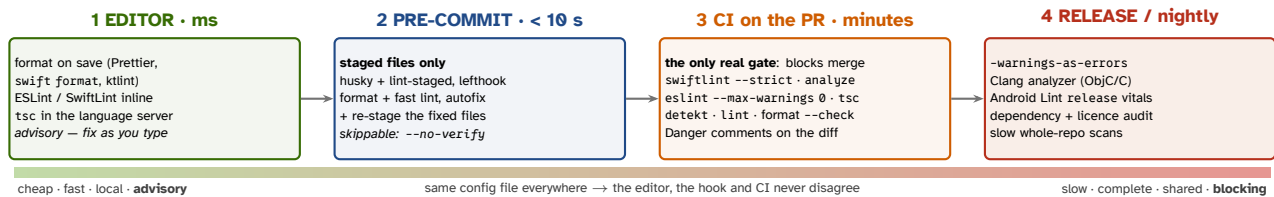


In one line: A linter is a **program that reads your source** and reports patterns; **syntax-only** rules (SwiftLint lint, ESLint core, ktlint) are fast and run on every save, **type-aware** ones (SwiftLint analyze, typescript-eslint typed rules, detekt with type resolution) need a compile and run in CI. A **formatter** rewrites layout and ends style debate. What works is the **pipeline**: the right check at the cheapest stage, one **blocking** gate in CI, a **baseline** for old code, and a written **suppression policy**.



Swift / iOS

- SwiftLint** (.swiftlint.yml): disabled_rules, opt_in_rules, only_rules, analyzer_rules, custom_rules (a regex + match_kinds + message + severity); strict = warnings are errors. Runs as an **SPM build tool plugin** (package SwiftLintPlugins) or a run-script phase.
- swiftlint analyze --compiler-log-path xcodebuild.log: **analyzer rules** (unused_declaration, unused_import) use the *type-checked* AST, need the log of a **clean** build, and are much slower — CI only.
- Formatter: swift-format** (swiftlang; in the toolchain since Swift 6 / Xcode 16: swift format -i, swift format lint --strict, JSON .swift-format) vs **SwiftFormat** (Nick Lockwood, community, more rules, .swiftformat, --lint mode). Pick one; let SwiftLint skip style rules it already fixes.
- Compiler** (SE-0443, Swift 6.1): -warnings-as-errors -Wwarning DeprecatedDeclaration = all errors except one group; -Werror <group> the reverse; **last flag wins**; -print-diagnostic-groups shows the group name.
- Clang static analyzer** (Product → Analyze, or “Analyze During Build”): path-sensitive checks for **C / Obj-C** — leaks, null derefs, use-after-free. It does not analyze Swift.

Example — the two configs that matter

```
# .swiftlint.yml
opt_in_rules: [force_unwrapping, empty_count]
analyzer_rules: [unused_declaration, unused_import]
excluded: [Pods, .build, "**/Generated"]
custom_rules:
  no_print:
    regex: "\\bprint\\("
    match_kinds: [identifier]
    message: "Use Logger, not print()"
    severity: error

// eslint.config.js (flat config)
import { defineConfig } from 'eslint/config';
import { tseslint } from 'typescript-eslint';
import reactHooks from 'eslint-plugin-react-hooks';
import prettier from 'eslint-config-prettier';
export default defineConfig([
  tseslint.configs.recommendedTypeChecked,
  reactHooks.configs.flat.recommended,
  { languageOptions: { parserOptions: { projectService: true } },
    lintOptions: { reportUnusedDisableDirectives: 'error' },
    rules: { '@typescript-eslint/no-floating-promises': 'error' } },
  prettier, // LAST: turns off every style rule
]);
```

Example — the CI lint job (cheapest first, fail fast)

```
swift format lint --strict --recursive Sources
swiftlint lint --strict --reporter github-actions-logging
xcodebuild clean build -scheme App ... > xcodebuild.log
swiftlint analyze --strict --compiler-log-path xcodebuild.log
npx prettier --check . && npx tsc --noEmit
npx eslint . --max-warnings 0 # warnings fail too
./gradlew detekt lint # android/
npx danger ci # PR comments, fail()
```

Remember

Format on save · lint the staged files · gate in CI · baseline the past · every suppression names a rule and a reason.

JavaScript / TypeScript / React Native

- ESLint** flat config (eslint.config.js, default since ESLint 9): an array of config objects, later ones override earlier; plugins are imported objects, not strings.
- typescript-eslint** recommendedTypeChecked + projectService: true asks TypeScript for types: no-floating-promises (an un-awaited promise), no-misused-promises (async fn in onPress expecting void), switch-exhaustiveness-check. Cost: a TS program build before linting.
- eslint-plugin-react-hooks**: rules-of-hooks, exhaustive-deps + the **React Compiler** rules in recommended/recommended-latest (purity, refs, immutability, set-state-in-render, set-state-in-effect, static-components...; flat: configs.flat.recommended). eslint-plugin-react-compiler is superseded by it.
- Prettier** = formatter only; eslint-config-prettier last in the config disables ESLint's style rules, so the two never fight. CI: prettier --check.
- Types are the biggest static analyser**: tsc --noEmit in CI (Metro/Babel only strip types) — strict flags: see ts-config-and-practice.

Android (the RN app's other half)

tool	finds	run · old debt
Android Lint	manifest, resources, API level, a11y, perf	./gradlew lint · lint-baseline.xml
detekt	Kotlin smells, complexity, bugs	detekt · detektBaseline → baseline.xml
ktlint	Kotlin style; --format fixes	pre-commit + CI

PR bots, hooks, gates

- Danger** (JS; also Danger Swift): a dangerfile runs in CI with the PR (danger.github.pr) and posts message/warn/fail: “no CHANGELOG”, “PR > 500 lines”, “lockfile changed without package.json”, SwiftLint results inline. **Judgement rules** a linter can't express.
- Hooks**: husky (.husky/pre-commit) + **lint-staged** (per-glob commands on staged files) · **lefthook** (Go binary, one lefthook.yml, parallel, {staged_files}; no Node needed — good for iOS repos). Hooks are **convenience**: --no-verify skips them, so CI repeats every check.
- Ratchet**: turning a rule on in a big repo = thousands of hits. Baseline them (SwiftLint write_baseline/baseline, detekt, Android Lint), block only **new** violations, burn the baseline down.

Suppressions policy

Narrowest scope + rule id + reason: // swiftlint:disable:next force_cast · // eslint-disable-next-line no-console -- CLI output · @Suppress("MagicNumber"). Ban blanket swiftlint:disable all / bare eslint-disable; report **unused** directives (superfluous_disable_command, reportUnusedDisableDirectives); count them in CI.

Traps

- ! Warnings nobody fails on grow forever — a warning is either an **error** or **off**.
- ! Pre-commit as the gate: skippable, per-machine versions. **CI** is the gate.
- ! Running analyze / typed ESLint on every keystroke — slow; they need a build.
- ! ESLint and Prettier both formatting — endless diffs; one owner per concern.
- ! Fixing lint in a 3,000-file “cleanup PR” — ruins git blame; baseline + ratchet.

Related: rm-testing-typescript (strict flags) · flaky-tests-ci-tdd (CI) · signing-and-cicd · sanitizers-runtime-checkers (dynamic analysis) · code-smells-refactoring · modularization-spm